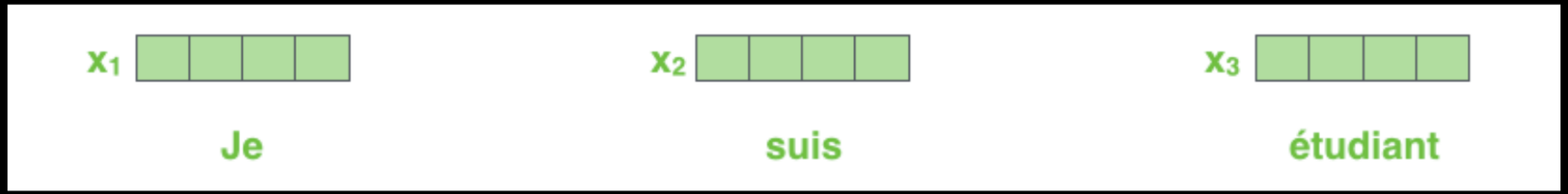
Архитектура трансформера

Алан-Барсаг Газзаев

Механизм внимания, предобученные модели и токенизация

14 ноября 2026

Место трансформера в обработке текста



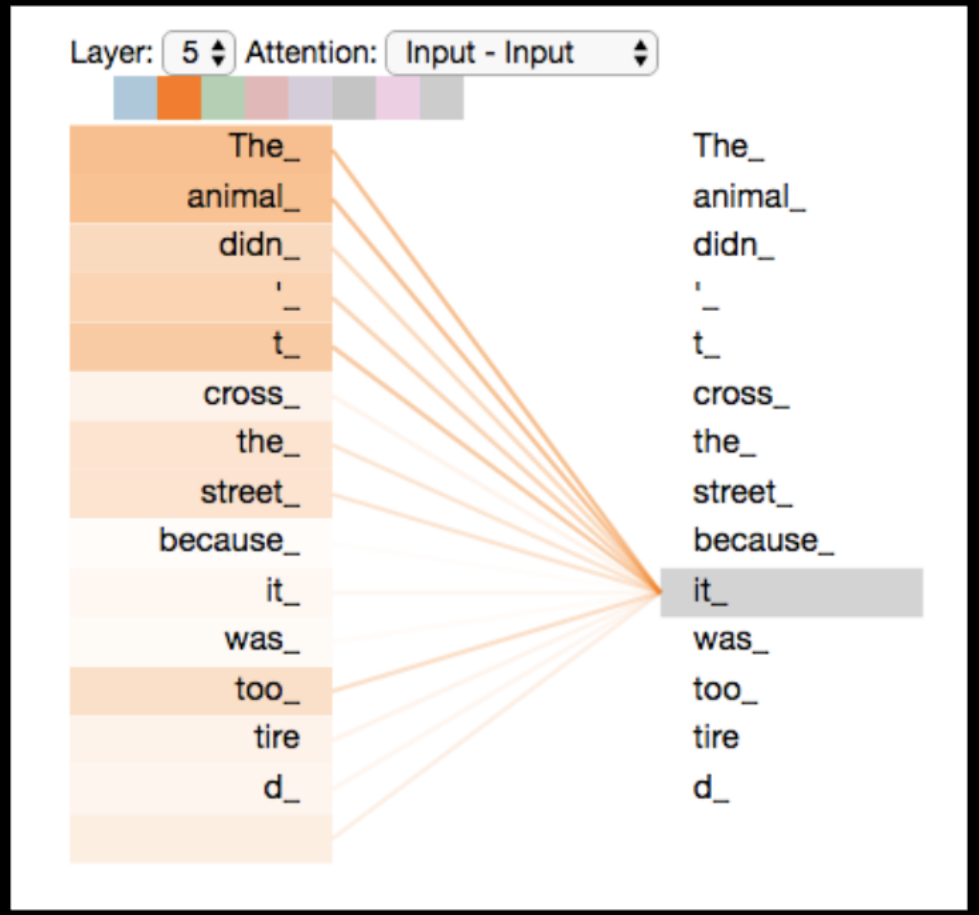
- Токенизатор преобразует строку в последовательность идентификаторов. Слой эмбеддингов сопоставляет каждому идентификатору обучаемый вектор.
- Трансформер обновляет представление каждой позиции с учётом доступного контекста. Выходная голова превращает представления в метки или вероятности токенов.
- Сегодня разберём вычисления внутри модели. Применение библиотек и дообучение перенесены в семинар.

Ограничения рекуррентных моделей

- В RNN состояние позиции зависит от предыдущего состояния: вычисления вдоль последовательности образуют цепочку.
- LSTM управляет сохранением и обновлением состояния с помощью вентилей. Это помогает обучению длинных зависимостей, но сохраняет последовательный расчёт.
- Свёртки обрабатывают позиции параллельно, однако для связи далёких токенов нужны дополнительные слои или расширение области обзора.
- Полное внимание напрямую связывает любые доступные позиции. Цена такого взаимодействия растёт квадратично с длиной текста.

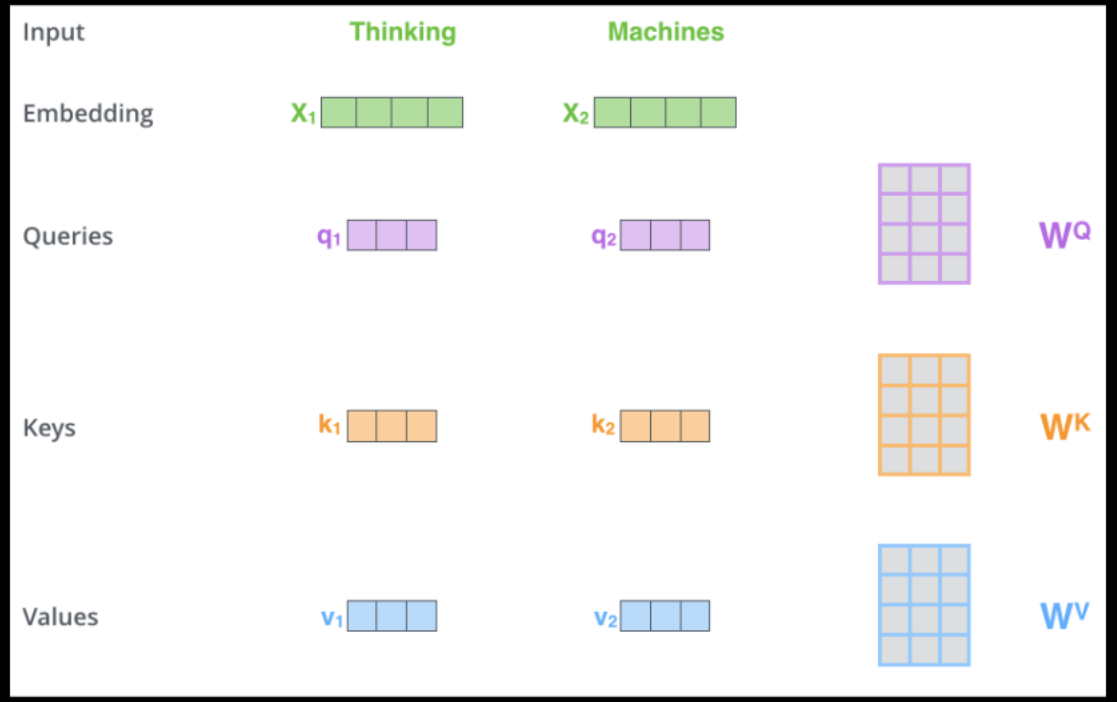
Контекст и неоднозначность

- Смысл слова зависит от окружения: «ключ от двери» и «ключ бьёт из земли» требуют разных представлений.
- Связь местоимения с существительным может проходить через несколько промежуточных слов.
- Механизм внимания вычисляет, какие позиции участвуют в обновлении текущего вектора и с какими весами.
- На схеме показан пример связи слов внутри одного предложения. Визуальные веса сами по себе ещё не доказывают причинное объяснение ответа.



Запросы, ключи и значения

- Каждая позиция получает три проекции исходного представления. Запрос Q участвует в поиске подходящих позиций, ключ K используется для сравнения, значение V переносит информацию.
- Матрицы проекций обучаются вместе с моделью. Это разные преобразования одного и того же входа.
- Названия «запрос» и «ключ» описывают роль в вычислении, а не поиск по внешней базе данных.

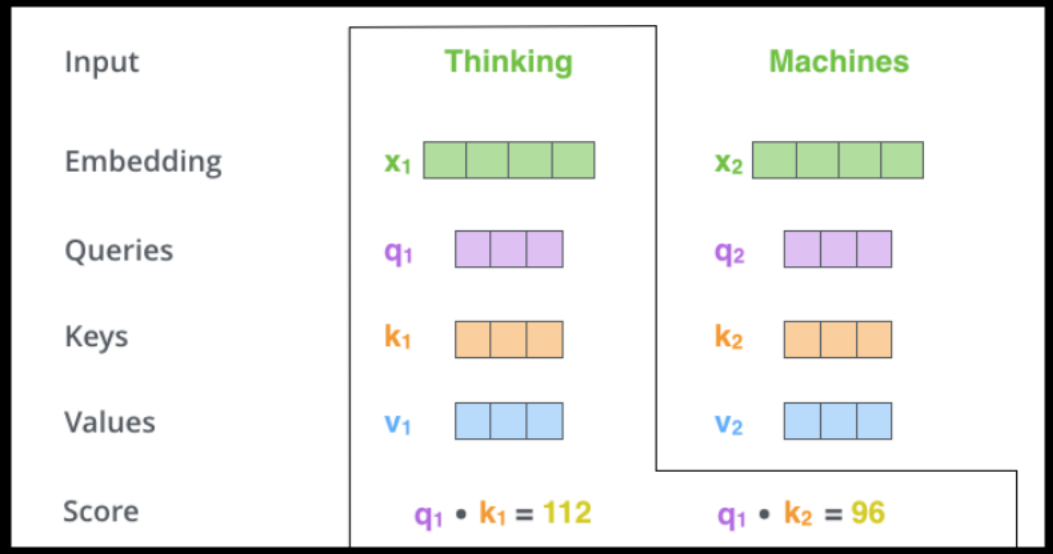


Размерности матриц внимания

$$X \in \mathbb{R}^{n \times d} \cdot Q = XW_Q \cdot K = XW_K \cdot V = XW_V$$
$$QK^T \in \mathbb{R}^{n \times n} \qquad AV \in \mathbb{R}^{n \times dv}$$

- Пусть X содержит n токенов, каждый описывается d_model признаками. Строки соответствуют позициям.
- W_Q и W_K имеют размер d_model × d_k; W_V имеет размер d_model × d_v.
- Произведение QK^T содержит оценку каждой пары позиций. После нормализации AV снова даёт по одному вектору на запрос.

Оценка совместимости позиций



- Для запроса q_i вычисляем скалярное произведение с каждым ключом k_j. Получаем числовые оценки s_ij.
- Большая оценка повышает относительный вес позиции j при обновлении позиции i.
- Проекции обучаемые, поэтому похожесть оценивается в пространстве конкретной головы внимания, а не по совпадению букв.

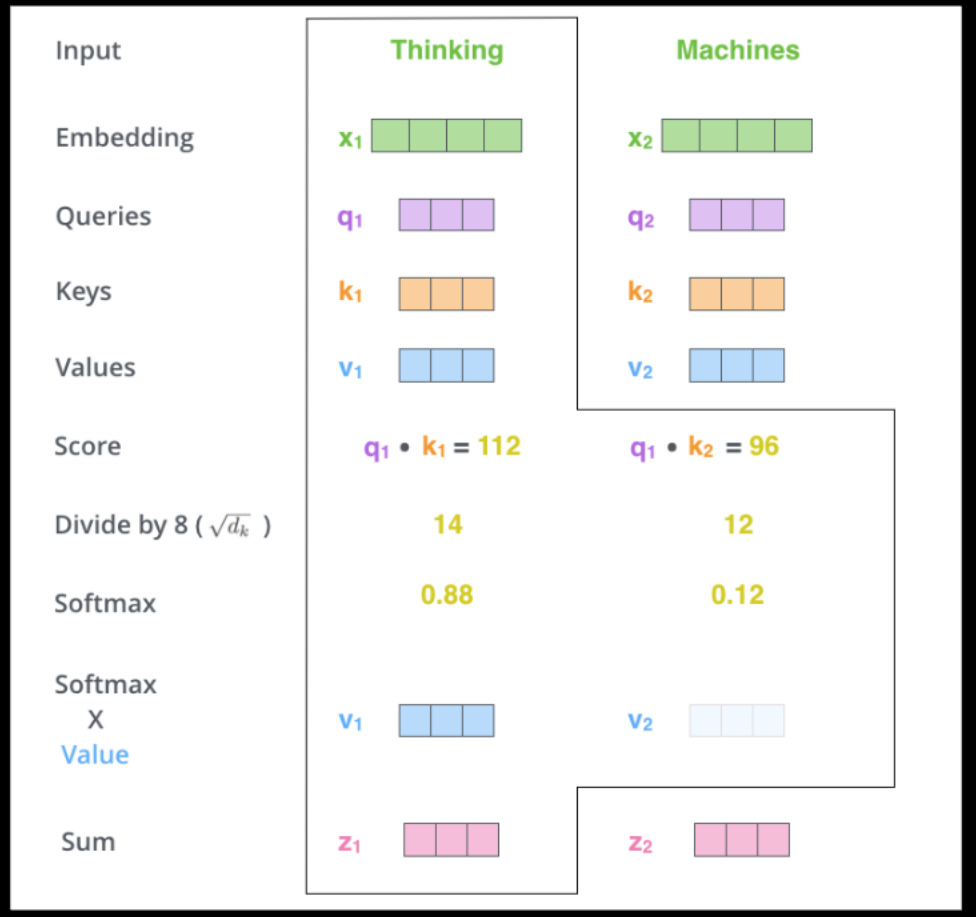
Масштабирование и нормализация

$$s_{ij} = (q_i \cdot k_j) / \sqrt{d_k}$$
$$\alpha_{ij} = \exp(s_{ij}) / \Sigma_l \exp(s_{il})$$

- Деление на $\sqrt{d_k}$ ограничивает рост масштаба скалярных произведений с увеличением размерности. При слишком больших оценках softmax становится резко сосредоточенным.
- Softmax применяется отдельно к каждой строке: все доступные ключи конкурируют за вес одного запроса.
- Полученные веса неотрицательны, их сумма по доступным позициям равна единице. Они зависят от всего набора ключей.

Взвешенная сумма значений

- После softmax каждое значение v_j умножается на вес α_ij. Затем эти векторы складываются.
- На выходе получается контекстное представление запроса i. Его размер определяется размерностью V.
- Веса выбирают сочетание информации, а сами значения содержат признаки, которые передаются дальше.

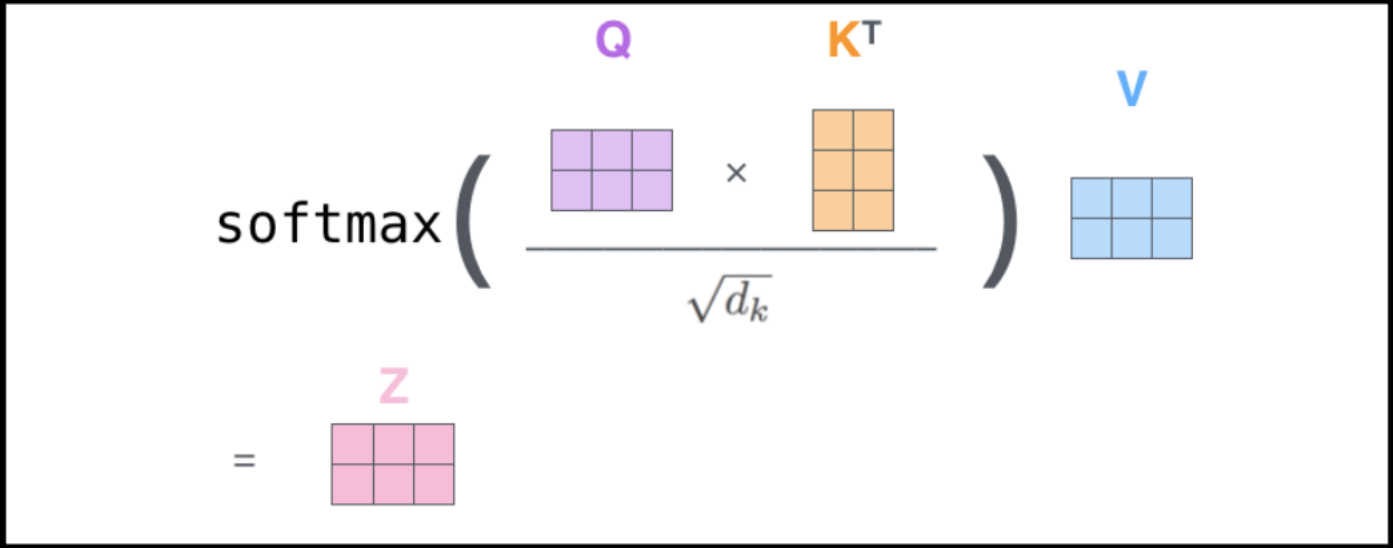


Численный пример внимания

$z \approx 0{,}401 \cdot (2, 0) + 0{,}198 \cdot (0, 2) + 0{,}401 \cdot (2, 2)$
$z \approx (1{,}604; 1{,}198)$

- При d_k = 2 возьмём q = (1, 0) и ключи $k_1 = (1, 0)$, $k_2 = (0, 1)$, $k_3 = (1, 1)$.
- После масштабирования оценки равны приблизительно (0,707; 0; 0,707). Softmax даёт веса (0,401; 0,198; 0,401).
- Пусть $v_1 = (2, 0)$, $v_2 = (0, 2)$, $v_3 = (2, 2)$. Итоговый вектор получается суммой трёх взвешенных значений.
- Это учебные числа, выбранные для ручного расчёта. Они не являются весами конкретной языковой модели.

Внимание для всей последовательности



- Матричная запись позволяет вычислять запросы всех позиций параллельно, когда входная последовательность уже известна.
- Матрица A показывает распределение весов по ключам для каждого запроса. Умножение на V собирает контекстные представления.
- Один и тот же токен может получить разные веса от разных запросов. Матрица внимания обычно несимметрична.

Маски внимания

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k} + M)V$$

$M_{ij} = 0$ для разрешённой связи; $-\infty$ для запрещённой

- Маска задаёт, какие связи разрешены. Запрещённые позиции получают $-\infty$ до softmax, поэтому их вес становится нулевым.
- Маска заполнения исключает служебные позиции PAD. Каузальная маска исключает будущие токены.
- MLM-маскирование заменяет часть входных токенов при обучении BERT. Это другая операция: она не задаёт треугольную матрицу доступности.

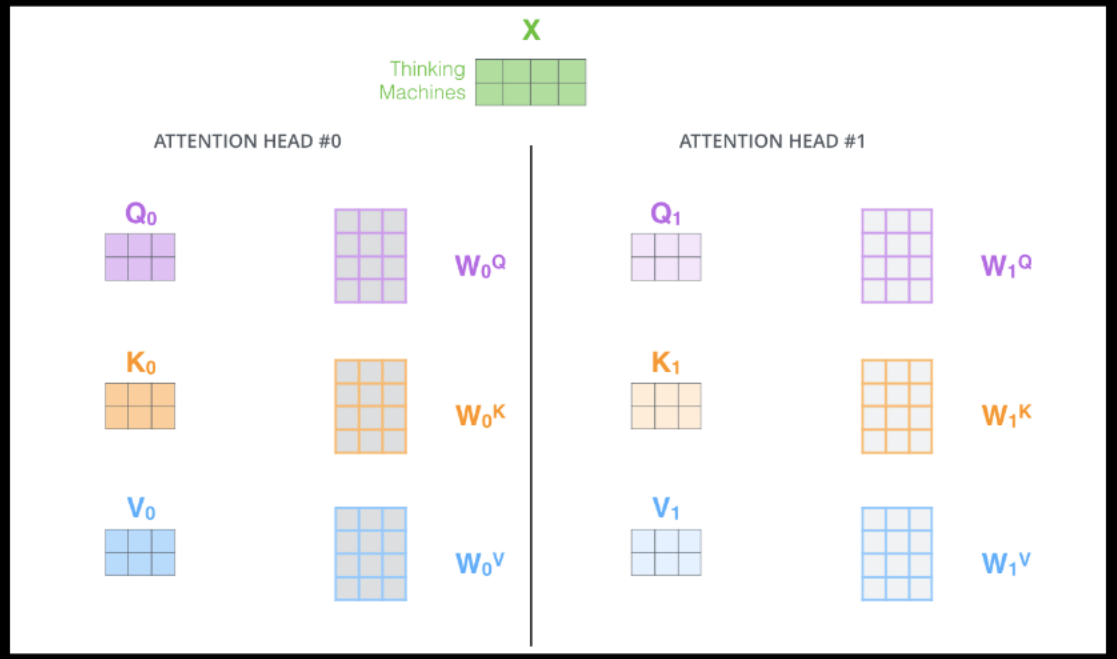
Каузальная доступность контекста

	ключи: 1	2	3	4
запрос 1	0	$-\infty$	$-\infty$	$-\infty$
запрос 2	0	0	$-\infty$	$-\infty$
запрос 3	0	0	0	$-\infty$
запрос 4	0	0	0	0

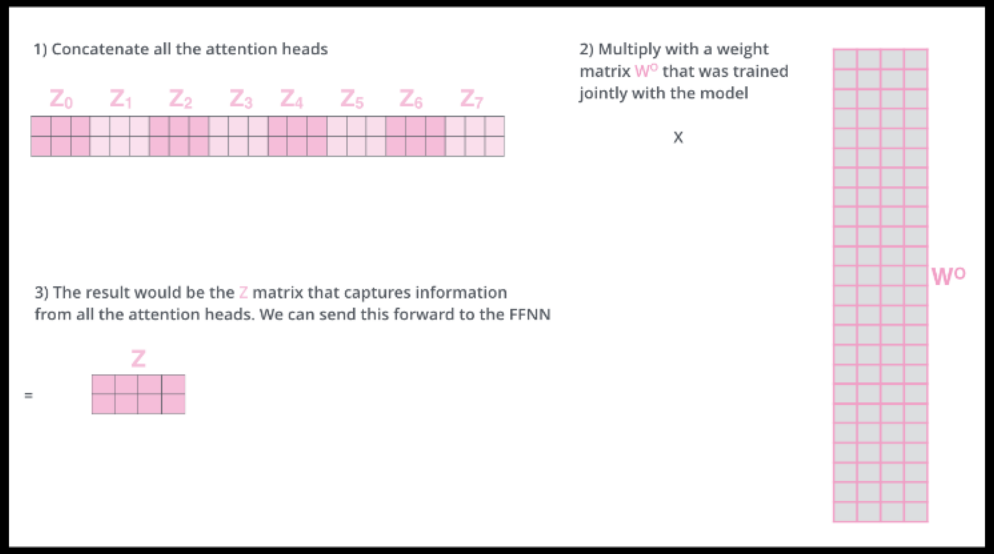
- Позиция i видит себя и предыдущие позиции, но не токены справа. Это соответствует предсказанию следующего токена по префиксу.
- При обучении известную последовательность можно обработать параллельно с маской. Целевые токены сдвинуты на одну позицию.
- При генерации следующий вход зависит от уже выбранного ответа, поэтому новые токены обычно появляются последовательно.

Многоголовое внимание

- Несколько голов используют собственные проекции Q, K и V. Они могут учитывать различные сочетания признаков и отношений.
- Число голов является параметром архитектуры. Фиксированное языковое назначение каждой голове заранее не задаётся.
- Размерность каждой головы обычно меньше d_model, поэтому несколько голов не означают несколько полноразмерных независимых моделей.

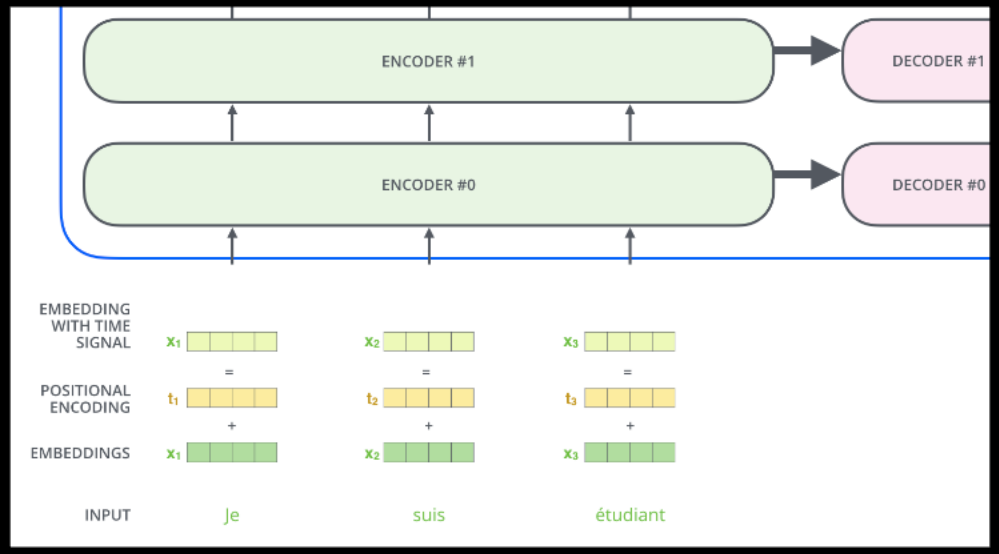


Объединение голов



- Выходы голов конкатенируются по признакам и проходят через обучаемую матрицу W_O.
- Выходная проекция смешивает результаты голов и возвращает требуемую размерность слоя.
- Головы рассчитываются параллельно. Последовательными остаются сами слои сети.

Позиционная информация



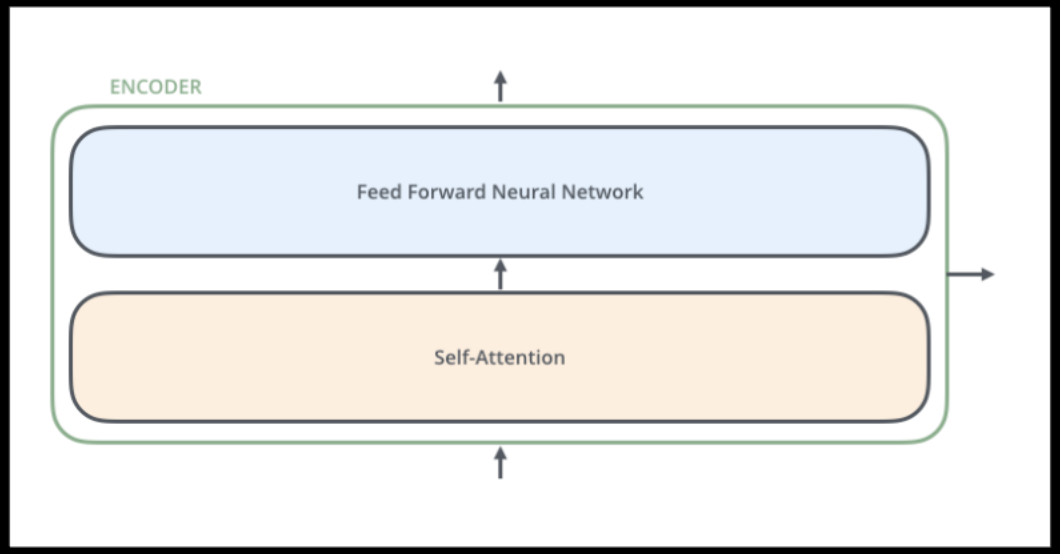
- Само внимание без позиционных признаков не кодирует порядок: перестановка входных векторов приводит к соответствующей перестановке выходов.
- Позиционные векторы добавляют информацию о месте токена. В исходном Transformer используются синусы и косинусы разных частот.
- Обучаемые позиции, относительные смещения и вращательные представления являются другими способами учесть порядок. Их нельзя механически заменять после предобучения.

Синусоидальное кодирование позиций

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$$
$$PE(pos, 2i + 1) = \cos(pos / 10000^{(2i/d_model)})$$

- pos обозначает номер позиции, i задаёт пару координат. Разные частоты позволяют представить порядок на нескольких масштабах.
- Синус и косинус занимают соседние координаты вектора. Полученный вектор складывается с эмбеддингом токена.
- Формула определена и за пределами обучающих длин, но это само по себе не гарантирует хорошую работу на более длинном контексте.

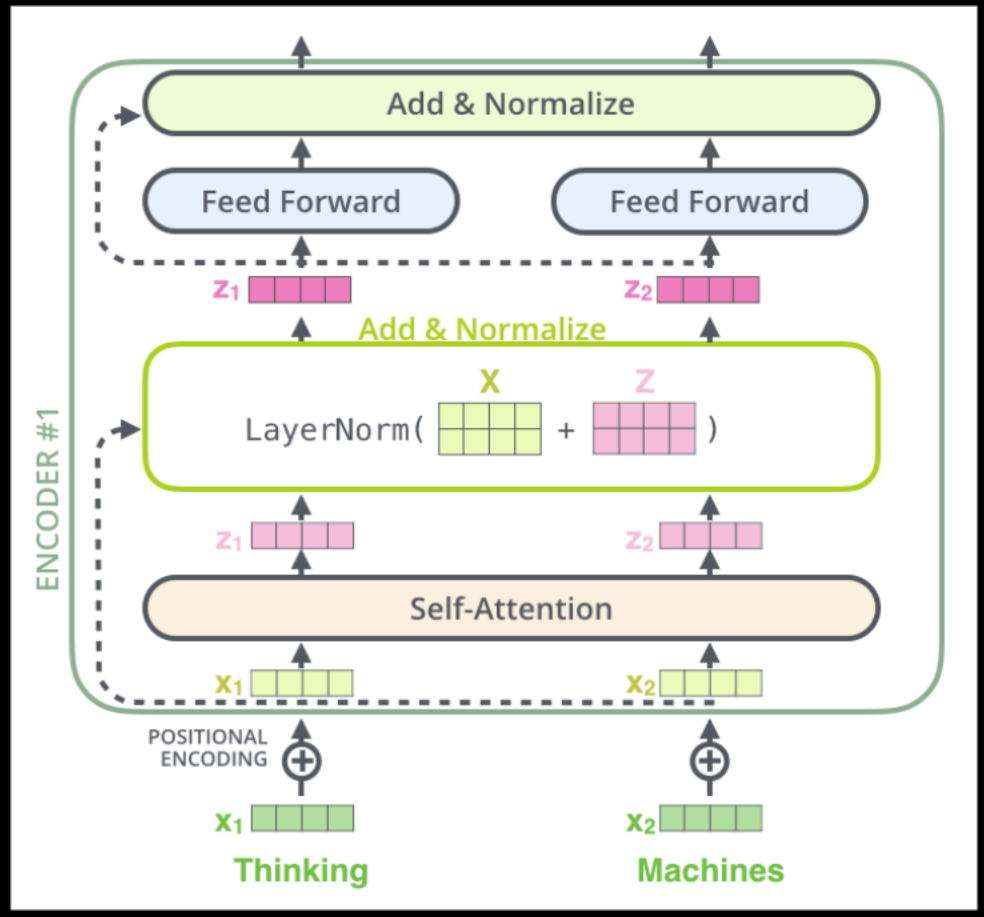
Полносвязное преобразование в блоке



- После обмена информацией между позициями сеть преобразует признаки каждого токена независимо.
- Одни и те же веса применяются ко всем позициям. Промежуточная размерность обычно больше размерности представления.
- Нелинейность позволяет строить более сложные признаки. В исходной архитектуре используется ReLU; у BERT используется GELU.

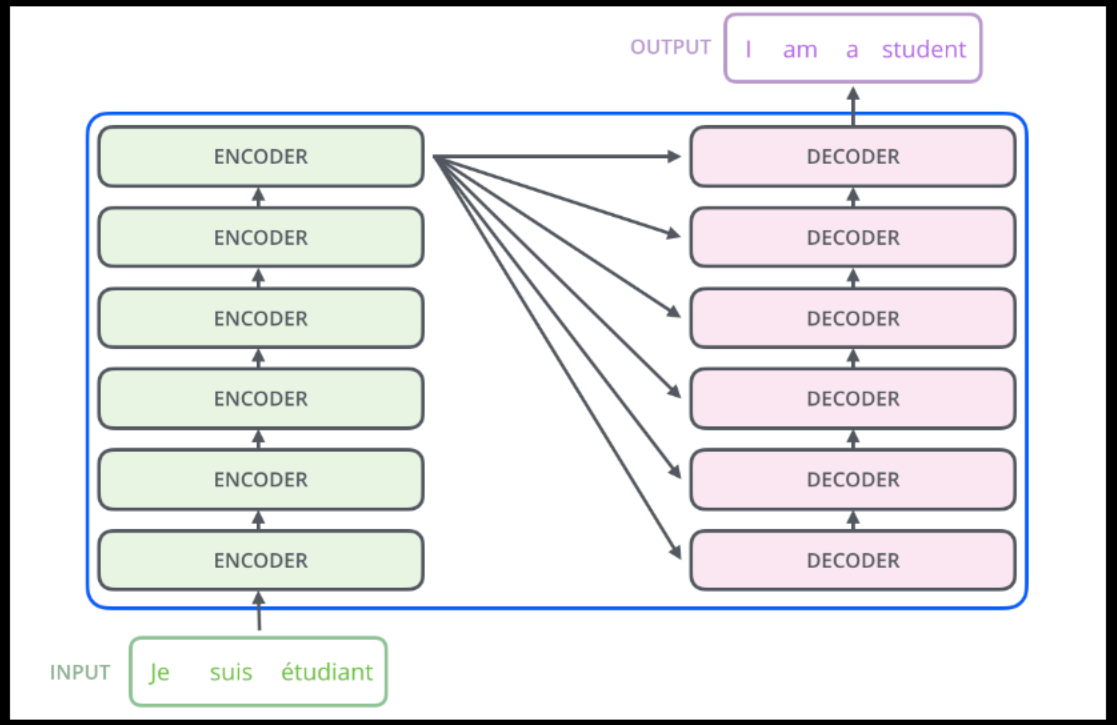
Остаточные связи и нормализация

- Остаточная связь складывает вход подслоя с его преобразованием. Она создаёт прямой путь передачи сигнала и градиента.
- LayerNorm нормализует признаки внутри представления отдельной позиции и использует обучаемые масштаб и смещение.
- В исходной статье нормализация стоит после остаточного сложения. Во многих последующих моделях используется нормализация перед подслоем.

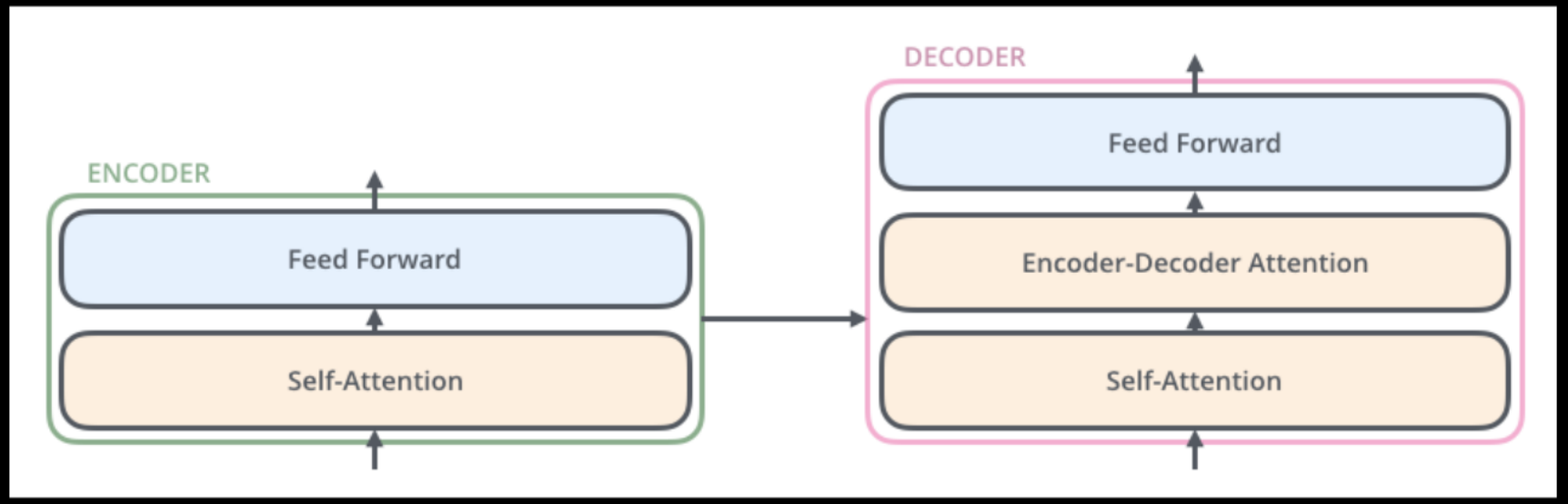


Полная архитектура трансформера

- Кодировщик преобразует исходную последовательность. Декодер генерирует целевую последовательность и обращается к выходам кодировщика.
- Слои внутри каждого стека имеют одинаковую структуру, но собственные параметры.
- Исходная архитектура решала задачу перевода. Для других задач часто используют только кодировщик или только декодер.



Перекрёстное внимание в декодере



- В собственном внимании Q, K и V получаются из одной последовательности.
- В перекрёстном внимании запросы приходят из декодера, а ключи и значения из выхода кодировщика.
- При переводе текущая позиция целевого текста выбирает информацию из исходного предложения. Матрица весов имеет размер n_target × n_source.

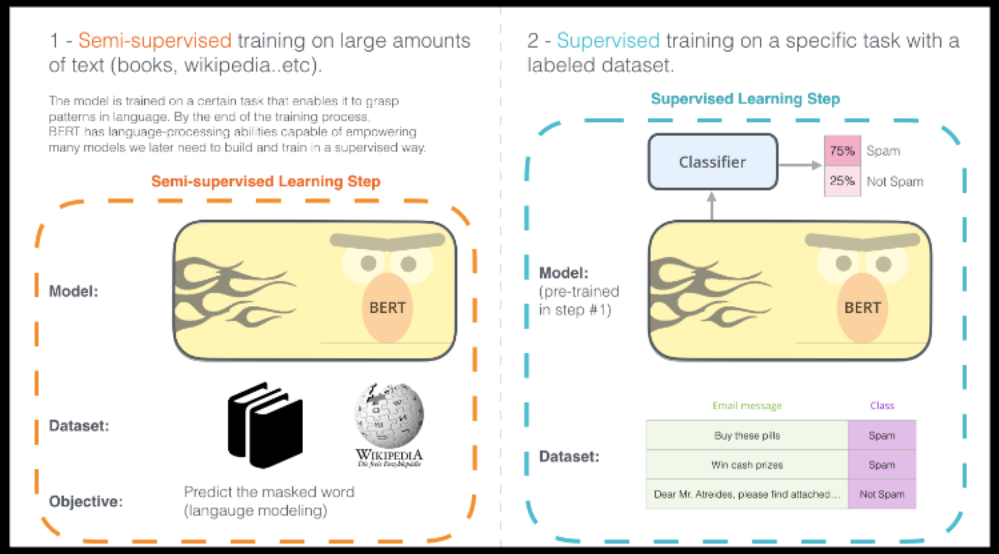
Три семейства архитектур

- Кодировщик читает доступный вход в обе стороны. Подходит для классификации, разметки токенов и извлечения ответа из текста.
- Декодер с каузальной маской предсказывает продолжение. Его используют для генерации и выполнения текстовых инструкций.
- Кодировщик–декодер отдельно представляет вход и генерирует выход с перекрёстным вниманием. Типичные задачи: перевод и суммаризация.
- Архитектура задаёт доступность контекста, а обучающая цель определяет, какие предсказания модель учится делать.

Вычислительная стоимость внимания

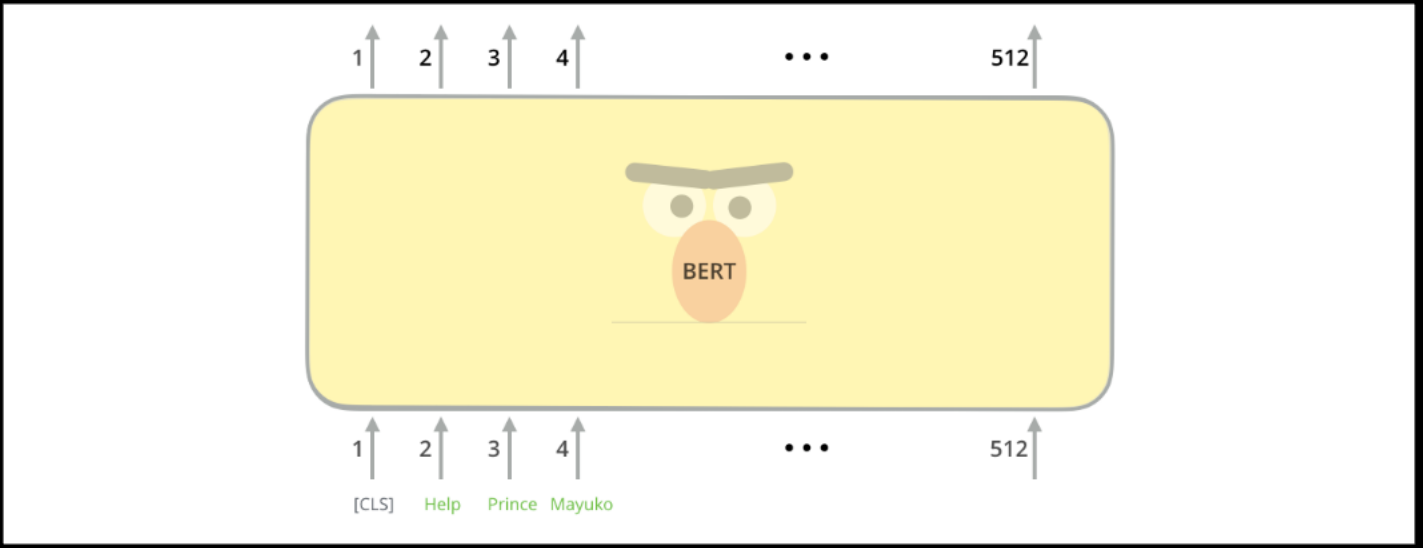
- Матрица полных попарных взаимодействий содержит n^2 элементов на голову. Удвоение длины увеличивает их число в четыре раза.
- Для обычного внимания вычисление QK^T и AV требует порядка n^2d операций. Дополнительные проекции и полносвязные слои тоже требуют ресурсов.
- Параллельный расчёт ускоряет обучение на подходящем оборудовании, но не делает длинный контекст бесплатным.
- Оптимизированные реализации могут не хранить всю матрицу в памяти. Это не отменяет числа взаимодействий в точном полном внимании.

Предобучение и адаптация модели



- При предобучении сеть получает общий обучающий сигнал из большого корпуса. У BERT это восстановление скрытых токенов и исходная задача связи предложений.
- При адаптации добавляют голову конкретной задачи и обучают её вместе со всей моделью либо выбранной частью параметров.
- Предобученные веса дают начальное представление языка, но качество на новой задаче нужно измерять отдельно.

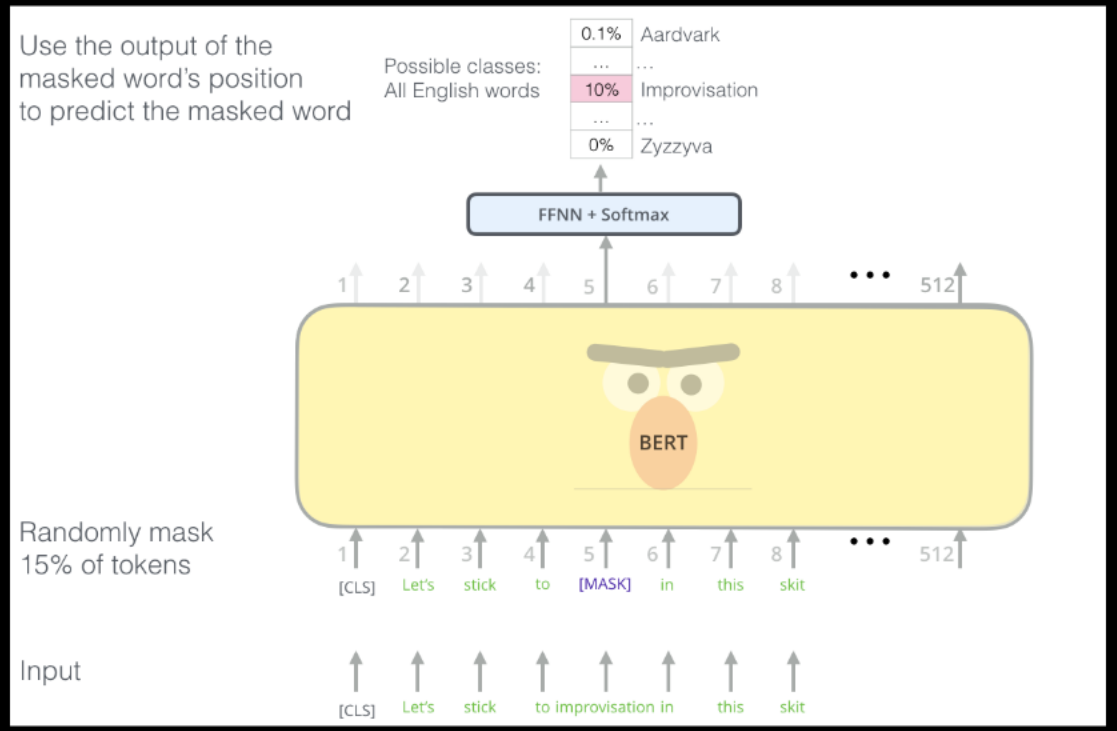
Двунаправленный кодировщик BERT



- BERT строит представление каждой позиции с учётом левого и правого контекста.
- Один и тот же токен получает разные выходные векторы в разных предложениях.
- Модель возвращает последовательность представлений. Для прикладного результата нужна голова задачи или специально выбранный способ агрегации.

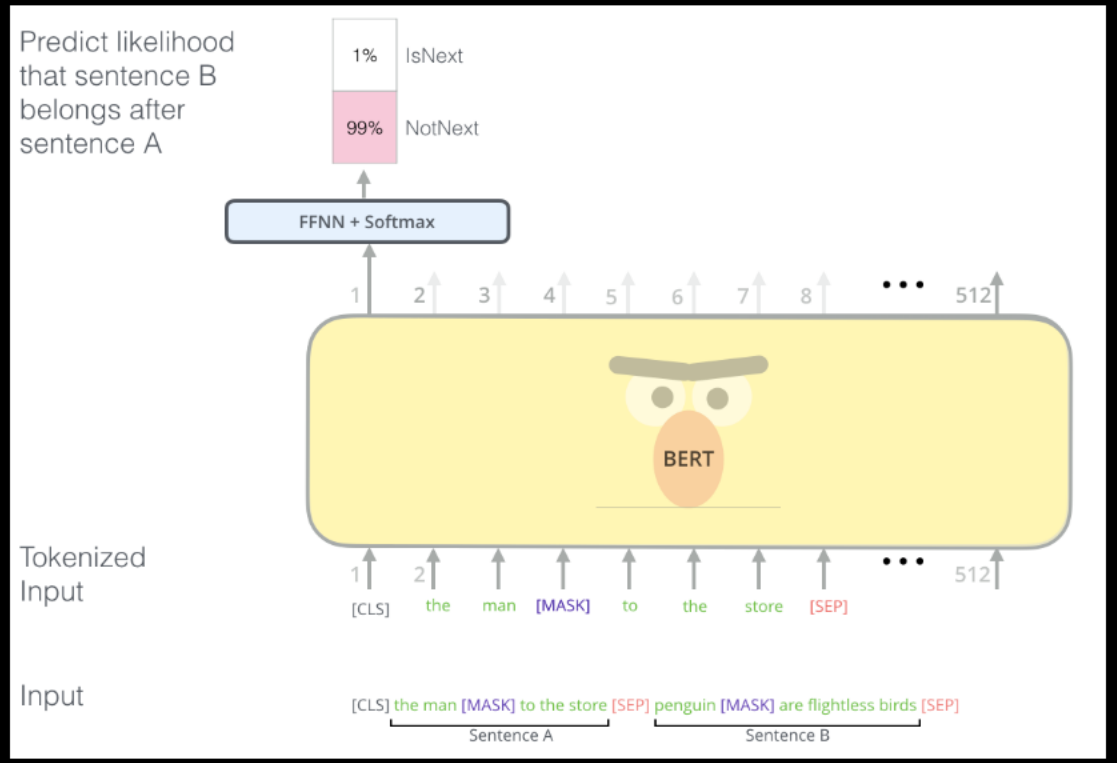
Восстановление скрытых токенов

- В исходном BERT для обучения выбирают 15% позиций. Среди выбранных 80% заменяют на [MASK], 10% на случайный токен, 10% оставляют без изменения.
- Потеря считается по выбранным позициям: модель должна восстановить исходные токены.
- Двунаправленный контекст помогает заполнить пропуск. В отличие от каузального обучения, модель не просто предсказывает следующий токен слева направо.



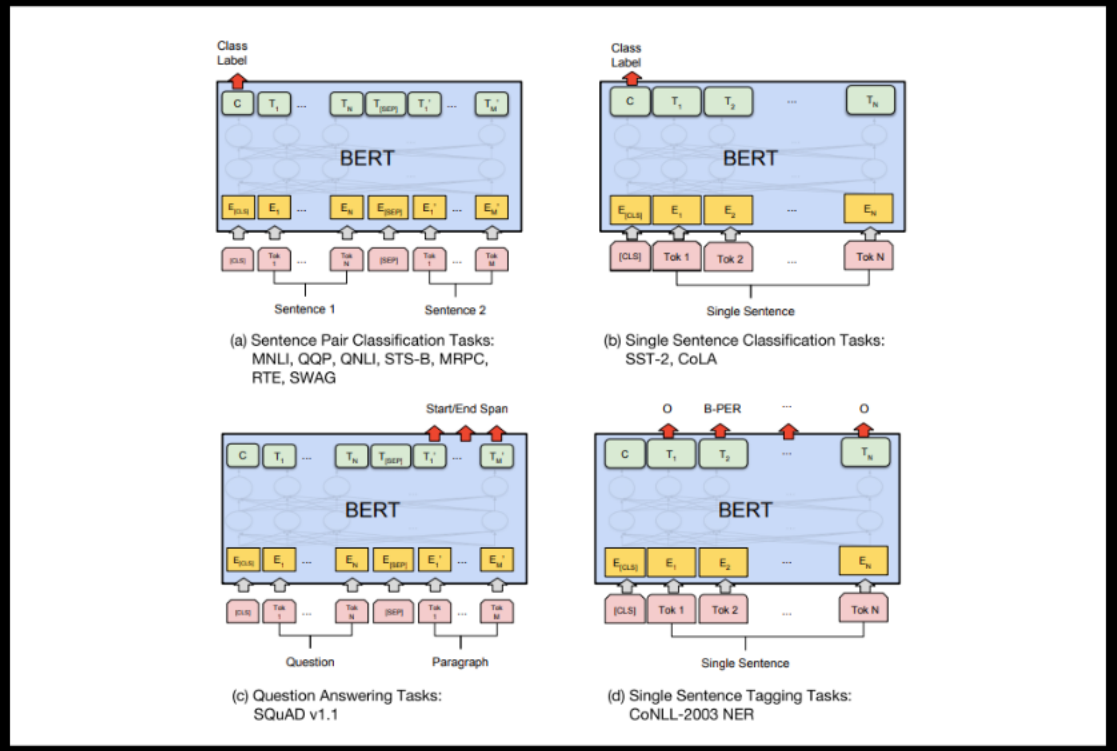
Пары предложений и вход BERT

- Вход может содержать два сегмента: [CLS] A [SEP] B [SEP]. К токенным и позиционным эмбеддингам добавляют эмбеддинг сегмента.
- В исходной задаче NSP модель различает настоящее продолжение и случайно выбранный второй сегмент.
- Этот обучающий сигнал не является универсальной проверкой логической совместимости. Позднейшие модели могут обходиться без NSP.



Головы прикладных задач

- Для классификации текста обучаемый классификатор использует представление [CLS] или другую согласованную агрегацию.
- Для разметки сущностей предсказание строится для токенов. Метки слов нужно согласовать с разбиением на подслова.
- Для извлечения ответа модель оценивает начало и конец фрагмента в контексте.
- Одинаковый кодировщик можно адаптировать к разным задачам, но головы и разметка у них различаются.



Развитие семейства BERT

- RoBERTa пересматривает режим предобучения: больше данных и обучения, динамическое маскирование, исключение NSP. Архитектурное семейство остаётся кодировочным.
- DeBERTa разделяет представления содержания и позиции в механизме внимания. Название не означает обычный генеративный декодер вместо BERT.
- Многоязычный BERT использует общие параметры и словарь для нескольких языков. Cased-версия исходного выпуска обучалась на 104 языках.
- Общий словарь позволяет перенос между языками, но качество и длина разбиения могут сильно различаться. Проверка русского текста остаётся необходимой.

Представления для семантического поиска

- Выходной вектор [CLS] из исходного BERT не следует автоматически считать хорошим эмбеддингом для сравнения предложений.
- Для семантического поиска важны обучающая цель, способ агрегации и нормализация. Специализированные модели обучаются сближать связанные тексты.
- При дообучении классификатора изменяются параметры под заданные метки. При извлечении признаков веса кодировщика можно зафиксировать.

Подсловная токенизация

- Словарь целых слов плохо покрывает редкие формы, опечатки и новые имена. Посимвольное представление делает последовательности длиннее.
- Подсловные методы хранят частые фрагменты и составляют редкие слова из нескольких единиц.
- Учебный пример: «переписывание» можно представить несколькими фрагментами. Конкретные границы определяются обученным словарём, а не правилами морфологии.
- Токенизатор и модель образуют согласованную пару: номер токена выбирает конкретную строку таблицы эмбеддингов.

Последовательное объединение пар

BPE: частоты соседних пар → упорядоченные слияния

- BPE начинает с небольших единиц и добавляет объединения наиболее частых соседних пар.
- Обучение определяет словарь и порядок слияний. При обработке нового текста применяются уже изученные правила.
- Учебная цепочка для часто встречающегося фрагмента: l + o + w, затем lo + w, затем low. Выбор слияния зависит от частот во всём корпусе.
- Byte-level BPE начинает с байтов. Это обеспечивает представимость строк, но один видимый символ может занимать несколько токенов.

Разбиение по словарю WordPiece

- WordPiece использует словарь подслов. В распространённом варианте разбор выбирает самый длинный подходящий фрагмент слева направо.
- В словаре BERT префикс ## обозначает продолжение слова. Он является служебной записью, а не частью исходного текста.
- Учебный пример при наличии нужных элементов: playing = play + ##ing. Реальный результат зависит от словаря и нормализации.
- WordPiece при применении использует словарь, а не последовательность BPE-слияний. При невозможности разбора возможен [UNK].

Токенизация средствами SentencePiece

- SentencePiece является библиотекой и форматом модели токенизатора. Она поддерживает, в частности, BPE и Unigram.
- Обучение возможно на исходных строках без предварительного разбиения на слова по языковым правилам.
- Пробел представляется специальным символом __, что помогает сохранить границы в последовательности токенов.
- Unigram выбирает разбиение по вероятностям элементов словаря. Поэтому SentencePiece нельзя считать третьим алгоритмом, полностью аналогичным BPE и WordPiece.

Типичные ошибки подготовки входа

- Токенизатор от другой модели может выдавать допустимые числа, которые соответствуют совсем другим строкам эмбеддингов.
- Изменение регистра, нормализации или специальных токенов влияет на вход относительно предобучения.
- Обрезка может удалить ответ, отрицание или важную часть вопроса. Предел измеряется в токенах выбранного токенизатора.
- Для NER нужна привязка подслов к словам, для ответов по фрагменту — к символьным позициям. Это проверяется на семинаре до обучения.

Вопросы для проверки понимания

- Почему суммы строк матрицы внимания равны единице, а суммы столбцов обычно нет?
- Что изменится в вычислении, если запретить модели смотреть вправо? Чем это отличается от замены слова на [MASK]?
- Откуда берутся Q, K и V при перекрёстном внимании?
- Почему нельзя заменить токенизатор у уже обученного BERT произвольным другим?
- Какие части модели нужно изменить для классификации предложений и для разметки сущностей?

Материалы лекции

- Vaswani et al. Attention Is All You Need; Devlin et al. BERT.
- Jay Alammar: The Illustrated Transformer; The Illustrated BERT. Оригинальные иллюстрации, лицензия CC BY-NC-SA 4.0.
- RoBERTa, DeBERTa и SentencePiece: статьи и официальные репозитории.
- Hugging Face Course: BPE и WordPiece.
- YSDA NLP course: github.com/yandexdataschool/nlp_course